

Custom ServiceNow Application: Library Services Management

Author: System Administrator

Scope: `u_borrow_request_mani` / borrow request 1

System Lifecycle State: 100% Configured, Automated, and Verified

1. Core Architecture & Schema (**What** $\rightarrow$ **Why** $\rightarrow$ **How** $\rightarrow$ **Outcome**)

Roles & Access Control

- **What:** Dedicated access roles for **Librarian** and **Student**.
- **Why:** To enforce strict data governance and protect transaction logs while ensuring a self-service experience for students.
- **How:** Created custom ACLs on the backend [Shared admin dashboard](#).
 - *Student Role:* Restricted to **Create** and **Read** permissions strictly on their own records.
 - *Librarian Role:* Full administrative **Read**, **Write**, and **Create** rights over all records across both tables.
- **Outcome:** Unauthorized users are completely locked out of editing transaction flows, creating a secure environment.

Data Tables & Field Configuration

- **Book Table (`u_book`)**
 - Tracks physical library assets. Key fields include Title, Author, ISBN, and Status (*Available*, *Borrowed*, *Maintenance*).
 - **Advanced Logic:** Configured a dynamic **Reference Qualifier** displaying only assets where **Status = Available** during a new request session.
- **Borrow Request Table (`u_borrow_request_mani`)**
 - Manages the student transactional lifecycle.
 - **Form Behaviors:** The *Borrower* auto-fills with the active session user, and *Borrow Date* defaults to the system current date.

UI Policy Setup

- **What:** Dynamic form visibility constraint on the **Return Date** field.
- **Why:** To prevent data clutter and ensure accurate data collection only when a book physically leaves the library.
- **How:** Set a UI Policy monitoring the request state.
- **Outcome:** The **Return Date** remains completely hidden/optional during the **Requested** and **Approved** phases, and instantly becomes **Visible and Mandatory** the absolute moment the status changes to **Issued**.

2. Process Automation & Real-World Testing

Flow Designer Engine

The automated backend engine handles a strict, real-world state machine tracking sequence:

Requested $\rightarrow$ Approved $\rightarrow$ Issued $\rightarrow$ Returned

1. **Submission & Routing:** Inserting a new record triggers the flow, instantly generating a role-based approval request for the Librarian group while shooting a confirmation email to the student.
2. **Fulfillment Tasking:** Upon approval, a **Catalog Task (sc_task)** is deployed for the librarians to physically locate and hand over the asset.
3. **State Syncing:** Completing the task shifts the parent request status to **Issued** (triggering the UI Policy) and flips the underlying book asset's status to **Borrowed**.

Quality Validation & Test Scenarios

To validate edge cases and ensure system resilience, the following use cases were verified:

- **Simultaneous Multi-Student Requests:** If two students attempt to request the exact same book, the system's *Reference Qualifier* instantly removes the book from the available search pool the second the first request is officially marked *Issued*, preventing double-booking bottlenecks.
- **Overdue Returns & Lifecycle Verification:** Manually updating a test transaction directly inside the backend data tables successfully verified that moving a state to *Returned* flags the book asset back to *Available* instantaneously, clearing it for the next user.

3. Governance, Reporting, & Insights

Reporting Engine

- **Report Name:** "Most Borrowed Books"
- **Visualization & Config:** Built directly in the ServiceNow Report Designer View to expose transaction analytics and look for bottlenecks.
- **Active Criteria Filter:** Strictly isolated to records matching **status | is | approved** to ensure only legitimate, verified cycles are calculated.

4. Portal User Experience Walkthrough

User Navigation Path

Home $\rightarrow$ Service Catalog $\rightarrow$ Library Services $\rightarrow$ Submit Borrow Request

Role-Based Narration Stream

- **The Student Perspective:** *“A student accesses the user-facing Service Portal, navigates to Library Services, and searches for an available title. They populate the form and hit submit. The session context auto-fills their credentials, generating an immediate backend record with a state of **Requested**.”*
- **The Librarian Perspective:** *“The librarian receives an automated notification dashboard widget item. They review the timeline, issue the approval, clear the physical fulfillment task, and hand over the asset. The platform dynamically handles the state transitions and mandates the return metrics until final drop-off.”*

5. Scalability & Future Roadmap

To transition this proof-of-concept into an enterprise-grade academic solution, the following enhancements are planned:

- **Automated Notifications:** Expand the Flow Designer layout to include automated time-delayed checks. If a book passes its mandatory **Return Date**, the engine will trigger escalation email/SMS reminders directly to the borrower.
- **Digital Catalog API Integrations:** Integrate with external open-source library systems (e.g., WorldCat or Google Books API) to automatically pull rich metadata, book covers, and author biographies into the **u_book** table using standard ISBN inputs.
- **Enhanced Analytical Dashboards:** Deploy a centralized interactive workspace dashboard tracking historical borrow frequencies, high-risk overdue students, and asset wear-and-tear metrics.
- **Academic Resource Expansion:** Scale the underlying schema structure to manage additional high-demand campus operations—such as lab equipment checkouts, study room bookings, and digital research license distribution.